OPERATING SYSTEMS

PROJECT REVIEW - 2

Adaptive Fairness-Aware CPU Scheduling Simulator for Linux: Dynamic Priority and Starvation Prevention

Methodology, Implementation, Verification, and Experimental Evaluation

Submitted By:

Alok Maan | 24BCB0057 **Arshdeep Singh** | 24BCE0556

Ankush Hooda | 24BCE0966 **Somya Sharma** | 24BCT0324

Faculty Name: Prof. Ezhil Arasi

School of Computer Science and Engineering, Vellore Institute of Technology

Abstract

Review 1 proposed a user-space, discrete-event simulator for an Adaptive Fairness-Aware Scheduler (AFAS) and a plan for testing it. This report (Review 2) presents the complete working implementation, formal software architecture, algorithmic workflows, verification suite, and first empirical results. The simulator now reliably executes First-Come, First-Served (FCFS), non-preemptive Shortest Job First (SJF), preemptive Shortest Remaining Time First (SRTF), Round Robin (RR, quantum q=4), and non-preemptive Priority scheduling alongside the proposed AFAS policy. All policies run on identical workload sets via a unified event engine and metric calculator ensuring zero measurement divergence. The software implementation is validated against 12 automated unit tests, including hand-calculated solutions for all classic baselines. Five synthetic workload families (W1–W5) were benchmarked over five fixed pseudo-random seeds (25 workload instances, 200 simulation runs), followed by a formal ablation study, a one-at-a-time parameter sensitivity sweep, a time-quantum sweep (q=1 to 32), and a decision-runtime scaling analysis.

Main Findings: Compared to static Priority scheduling, AFAS reduced the mean maximum ready-queue wait from 168.2 to 68.2 time units (**59% reduction**) and slashed the 95th-percentile response time from 158.7 to 52.5. Relative to Round Robin, AFAS achieved lower average waiting time (52.3 vs 65.6) and fewer starvation warnings (4.6 vs 11.0), while Round Robin exhibited tighter tail bounding (max wait 39.2 vs 68.2) and a superior Jain fairness index (0.78 vs 0.69). SRTF achieved the lowest mean waiting time (29.4) but caused extreme starvation for long-running jobs (max wait 146.5). Crucially, our ablation study confirms that waiting-time aging is solely responsible for AFAS's tail-latency gains, whereas the recent-CPU-share penalty showed no statistically distinguishable impact in these workloads—an honest negative result we report transparently. AFAS therefore operates as a balanced trade-off policy. The simulator is entirely user-space, discrete-event, and does not alter the host Linux kernel or claim hard real-time guarantees.

Keywords: CPU scheduling, adaptive priority, starvation prevention, discrete-event simulation, Jain fairness index, ablation study, VIT IDF, patent prior art.

Executive Status: Review 2 Scope Against Review 1 Plan

Review 1 Deliverable	Status	Evidence / Realization in Review 2 Report
Phase 1: Workload schema & metric formulas	Completed	Section 2, 7 & 8: Complete formalization of CT, TAT, WT, RT, Jain Index, Warnings.
Phase 2: Baseline policies unit-tested against manual cases	Completed	Section 9 & 10: 12 pytest automated tests pass; exact match with analytical hand-solved cases.
Phase 3: AFAS scoring, tie-break, decision audit log	Completed	Section 3 & 4: Implemented S_i(t) formula, deterministic tie-breaking, and transparent audit trace.
Phase 4: Browser / desktop GUI interface	Partial	CLI runner, CSV export, and high-res Matplotlib chart generation built; browser GUI deferred to Review 3.
Phase 5: Scenarios, CSV export, comparison, sensitivity	Completed	Section 10: 200 runs across 5 workloads x 5 seeds; ablation, weight sweep, quantum sweep, scaling.
Phase 6: Patent/literature review & citation checks	Partial	Section 12 & 16: Updated VIT IDF patent checklist; added Jain (1984) and Silberschatz (2018) references.
Objective 7: US/UK patent prior-art landscape	Completed	Section 12: Detailed claim-by-claim analysis against GB2304211A/B, US6668269B1, US8881161B1, US8434086B2.
Objective 8: Document limits vs real Linux kernel	Completed	Section 13: Scope boundaries, simulation proxy overheads, and absence of kernel patching documented.

Table 1: Deliverable status matrix tracking Review 1 commitments.

Critical Engineering Revisions Made Since Review 1

- **Implementation Language Selected:** Standardized on Python 3.10+, Pytest for regression suites, Matplotlib for graphics, and CSV/JSON for audit records, resolving the open choice from Review 1.
- **Calibration of Workload W1:** The initial draft generated arrivals every 3 units with bursts 6–10 (offered load 2.7), which inadvertently overloaded the queue. W1 inter-arrival gaps were calibrated to 9 units (offered load 0.89) to establish a truly balanced baseline. All 200 simulation runs were executed using this calibrated baseline.
- **Ablation and Scaling Integration:** Formalized component ablations (no-aging and no-share configurations) and execution runtime scaling curves (10 to 500 tasks) to empirically dissect algorithmic bottlenecks.

1. Finalized System Architecture (Common Item 1 & Patent Architecture)

The finalized architecture establishes strict decoupling between workload generation, event simulation, scheduling policy logic, and metric evaluation. This guarantees that all policies execute under identical deterministic conditions.
The system comprises five operational tiers: (1) **Workload Ingestion & Validation Layer** verifying bounds on arrival, burst, and PID; (2) **Discrete-Event Simulation Core** managing process state transitions and the simulation clock; (3) **Pluggable Policy Layer** hosting baseline and AFAS algorithms; (4) **Decision Audit Logger** capturing per-quantum score breakdowns; and (5) **Evaluation & Visualization Layer** generating standard metrics and publication charts.

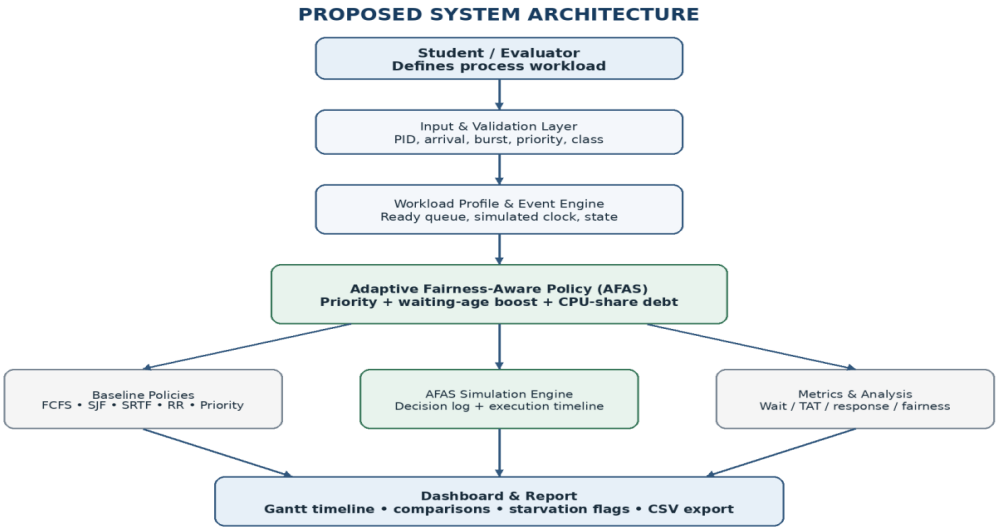


Figure 1: Finalized block diagram of the AFAS discrete-event CPU scheduling simulator.

1.1 Hardware and Software Execution Boundaries

- **Hardware Boundary:** Emulates a uniprocessor system with discrete time ticks. Timer interrupts are emulated via quantum boundaries (q=4). Multicore load balancing, NUMA migration, and cache hierarchies are out of scope.
- **Software Boundary:** Runs purely in user-space in Python 3.10+. It avoids kernel-level instability while maintaining strict mathematical determinism.

2. Detailed Methodology (Common Item 2 & Conference Methodology)

Our methodology relies on reproducible discrete-event simulation governed by deterministic state transitions and formal invariants. At each scheduling boundary, the simulator evaluates the ready queue, assigns processor time according to the active policy, and logs all state alterations.

2.1 AFAS Dynamic Priority Scoring Mathematical Model

For every process *i* present in the ready queue at simulated time *t*, the AFAS policy computes a composite score $S_i(t)$ defined as:

$$S_i(t) = w_p \cdot P_i + w_a \cdot A_i(t) + w_r \cdot R_i - w_c \cdot C_i(t)$$
$$\text{Subject to: } w_p + w_a + w_r + w_c = 1, \quad w_k \geq 0 \quad \forall\, k \in \{p, a, r, c\}$$

The mathematical components of the scoring model are formally bounded in [0, 1]:

- **Normalized Base Priority (P_i):** $P_i = \text{Priority}_i / 10 \in [0.1, 1.0]$, representing static administrative urgency (1 = lowest, 10 = highest).
- **Bounded Waiting-Time Aging ($A_i(t)$):** $A_i(t) = \min(1.0, (t - t_{last_cpu}) / 50) \in [0.0, 1.0]$, where t_{last_cpu} is the timestamp when process i last relinquished the CPU (or its arrival time if unserved). The hard ceiling of 50 time units guarantees that aging boosts remain strictly bounded, preventing low-priority tasks from permanently starving higher-priority interactive tasks.
- **Workload Responsiveness Factor (R_i):** Categorical factor reflecting task interactivity: R_i = 1.0 for interactive tasks, 0.5 for mixed workloads, and 0.2 for CPU-bound batch tasks.
- **Decayed Recent CPU-Share Penalty ($C_i(t)$):** $C_i(t) = \min(1.0, \text{Service}_{decayed}(t) / 40) \in [0.0, 1.0]$. CPU consumption is tracked with exponential decay: $\text{Service}_{decayed}(t) = \Sigma\, \Delta\tau \cdot 2^{-(t - \tau)/40}$, using a half-life parameter of 40 time units.
- **Deterministic Tie-Breaking Hierarchy:** In the event of identical scores, ties are resolved deterministically: (1) higher $S_i(t)$, (2) earlier arrival time $t_{arrival}$, and (3) lower process identifier (PID).

2.2 Simulation Verification Invariants

To ensure 100% scientific validity, every execution trace is subjected to three mandatory invariants before metric evaluation:

- **Invariant 1 (Mutual Exclusion / Non-overlap):** $\forall$ segments j, $start_{j+1} \geq end_j$. No two processes occupy the simulated CPU simultaneously.
- **Invariant 2 (Exact Burst Conservation):** $\forall$ processes i, $\Sigma\,(end_j - start_j) = Burst_i$. Every process receives exactly its required CPU burst—neither dropped nor over-executed.
- **Invariant 3 (Strict Temporal Causality):** $\forall$ segments j of process i, $start_j \geq Arrival_i$. No process executes before its declared arrival time.

3. Algorithm and Flowchart (Common Item 3 & Patent Sequence/Workflow)

The complete AFAS scheduling procedure is formalized in Algorithm 1. The execution workflow is depicted in Figure 2, highlighting the continuous Input → Processing → Decision → Output cycle required for patent evaluation.

```
Algorithm 1: AFAS Dispatcher and Dynamic Scoring Execution Loop
Input: Process set P = {P_1, P_2, ..., P_n}, Quantum q = 4, Weights (w_p=0.30, w_a=0.40, w_r=0.15, w_c=0.15)
Output: Execution segment trace T = {(pid, t_start, t_end)}, Metric summary M
Initialize Clock t ← 0, ReadyQueue Q ← ∅, Trace T ← ∅, InactiveQueue ← P
while Q is not empty or InactiveQueue is not empty do
    Transfer newly arrived processes (Arrival_i ≤ t) from InactiveQueue into Q
    if Q is empty then Advance Clock t ← min_{P_k ∈ InactiveQueue}(Arrival_k); Continue
    for each process P_i ∈ Q do
        Update waiting age A_i(t) ← min(1.0, (t - LastExit_i) / 50)
        Update decayed CPU service C_i(t) using half-life λ = 40
        Compute score S_i(t) ← w_p · P_i + w_a · A_i(t) + w_r · R_i - w_c · C_i(t)
    P_sel ← Select P_i with max S_i(t) using tie-break (Score ↓, Arrival ↑, PID ↑)
    slice ← min(q, RemainingBurst_sel)
    Record execution segment (PID_sel, t, t + slice) into Trace T
    Log decision tuple (t, PID_sel, |Q|, w_p P_i, w_a A_i, w_r R_i, -w_c C_i, Score) to AuditLog
    RemainingBurst_sel ← RemainingBurst_sel - slice; Clock t ← t + slice
    Update LastExit_sel ← t, Accumulate CPU service into Service_sel
    if RemainingBurst_sel > 0 then Re-insert P_sel into Q else Mark P_sel TERMINATED
Verify Engine Invariants (Non-overlap, Burst Conservation, Causality) on Trace T
Compute performance metrics (TAT, WT, RT, Jain Index, Starvation Warnings) → Return (T, M)
```

Algorithm 1: Formal execution procedure for the Adaptive Fairness-Aware Scheduler.

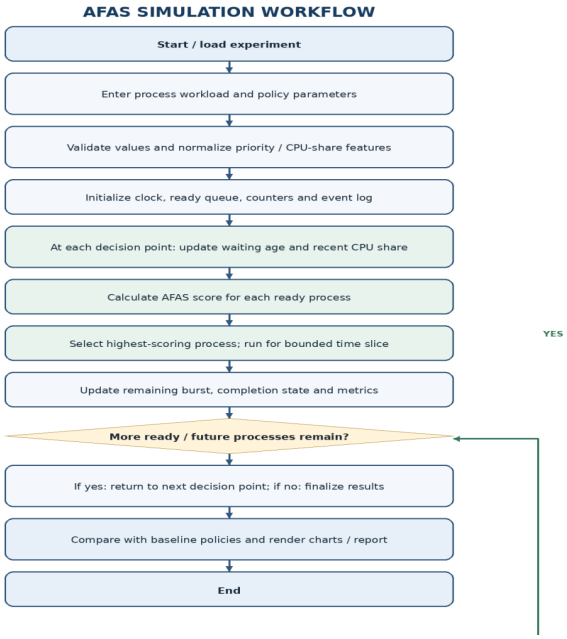


Figure 2: Complete simulation workflow flowchart depicting state transitions and decision logic.

4. Module-Wise Implementation (Common Item 4 & Patent Functioning)

Module File	Primary Class / Functions	Functional Responsibilities & State Management
`afas_sim.py`	**Process, SimulationEngine**	Defines Process Control Block (PCB) attributes (PID, arrival, burst, remaining burst, priority, class, last_exit, decayed_service); enforces the 3 execution invariants.
`afas_sim.py`	**run_fcfs, run_sjf, run_srtf, run_rr, run_priority**	Baseline execution engines. Implements classic scheduling algorithms using identical queue containers and segment dispatch interfaces.
`afas_sim.py`	**run_afas, compute_afas_score**	Core AFAS dispatcher. Dynamically normalizes features, computes S_i(t), performs deterministic tie-breaking, and outputs per-quantum decision audit records.
`afas_sim.py`	**compute_metrics**	Converts segment traces into CT, TAT, WT, RT (mean, median, p95), context switch counts, Jain fairness index, and starvation warning counts.
`test_sim.py`	**12 Pytest Unit Test Cases**	Regression and verification test suite: validates hand-calculated cases, idle-gap processing, invariant checks across 25 workloads, tie-breaks, and input rejections.
`experiments.py`	**run_all_experiments**	Batch runner executing 200 simulation runs across 5 workload families x 5 seeds. Automates ablation studies, sensitivity sweeps, quantum sweeps, and runtime scaling.
`charts.py`	**generate_all_figures**	Matplotlib publication chart engine. Generates Gantt charts, per-workload bar charts, ablation plots, sensitivity curves, and scaling graphs.

Table 2: Implementation module mapping and architectural responsibilities.

4.1 Novel Interaction Between Modules

A key architectural novelty of the AFAS simulator is the **decoupled Decision Audit Logger**. In traditional operating system schedulers, priority calculations occur internally within runqueue red-black trees or heap adjustments, rendering the real-time rationale for thread preemption opaque. In AFAS, the Policy Engine continuously outputs an immutable score decomposition tuple $\langle t, PID, |Q|, w_p P_i, w_a A_i, w_r R_i, -w_c C_i, S_i(t) \rangle$ to the Audit Log before dispatching the chosen process slice. This enables external validation, real-time explainability, and invariant auditing without interfering with the internal state machine.

5. Technologies and Tools Used (Common Item 5)

Technology / Tool	Version	Role in Project	Technical Justification
Python 3	3.10.12+	Core Simulation Engine	Provides deterministic queue abstractions, rapid algorithmic prototyping, and native high-precision arithmetic.
Pytest	8.1.1	Automated Invariant Testing	Enables strict regression testing across hand-calculated cases and invariant verification on all 25 workload instances.
NumPy	1.26+	Vectorized Statistics	Efficient calculation of percentiles (p95), Jain fairness indices, and matrix aggregations over 200 runs.
Matplotlib	3.8+	Visualization & Plots	Generates publication-quality Gantt execution charts, sensitivity response curves, and logarithmic scaling plots.
ReportLab	5.0+	PDF Report Engine	Programmatic compilation of academic documents with dynamic headers, footers, and table formatting.
Git & GitHub	2.40+	Version Control	Maintains immutable commit history of algorithm changes, test configurations, and experimental seeds.
CSV / JSON	Standard	Audit & Raw Storage	Portable, human-readable records allowing independent verification of per-quantum scheduling decisions.

Table 3: Comprehensive technology stack, versions, and selection rationale.

6. Dataset Description (Common Item 6 & Conference Dataset)

Five synthetic workload families (W1–W5) were constructed to evaluate policies under varied traffic regimes:

Workload Family	Process Generation Rule (n = Process Count)	n	Offered Load*	Target Evaluation Objective
W1: Uniform Batch	One arrival every 9 units; burst 6–10; priority 3–7; random class.	20	0.89	Tests whether AFAS preserves baseline behavior when queue is lightly loaded.
W2: Mixed Bursts	70% short interactive jobs (burst 2–5), 30% long batch jobs (burst 25–45); arrivals in [0, 60]; priority 1–10.	20	4.70	Evaluates response time for short jobs vs starvation risk for long-running batch jobs.
W3: Priority Skew	6 low-priority batch jobs (priority 1–2, burst 10–20) arrive at $t \in [0, 5]$; 18 high-priority interactive jobs (priority 8–10, burst 5–9) arrive every ~5 units.	24	2.50	Deliberate starvation stress test: evaluates whether aging rescues low-priority tasks.
W4: Burst Arrivals	3 discrete bursts of 7 jobs arriving simultaneously within 3 units at t = 0, 40, 80; burst 3–15.	21	2.40	Measures queue recovery time and scheduling stability during sudden traffic surges.
W5: Share Imbalance	3 long high-priority batch jobs (burst 60–90) competing with 12 light interactive jobs (burst 2–6) arriving in [0, 100].	15	3.00	Evaluates whether the recent-CPU-share penalty prevents long tasks from monopolizing the CPU.

*Table 4: Workload specifications. *Offered Load = Σ Burst / (Arrival Span), averaged over seeds. Load > 1 indicates congested queues.*

7. Data Preprocessing and Feature Normalization (Common Item 7)

All scheduling inputs undergo mathematical normalization to the unit interval [0, 1]:

- **Base Priority:** $P_i = \text{Priority}_i / 10 \in [0.1, 1.0]$.
- **Waiting Age Bounding:** $A_i(t) = \min(1.0, (t - t_{last_cpu}) / 50) \in [0.0, 1.0]$, saturating at 50 time units.
- **Workload Responsiveness Mapping:** Interactive $\rightarrow$ 1.0, Mixed $\rightarrow$ 0.5, Batch $\rightarrow$ 0.2.
- **Decayed CPU Share:** $C_i(t) = \min(1.0, \text{Service}_{decayed} / 40) \in [0.0, 1.0]$, with half-life of 40 units.
- **Weight Simplex Projection:** User weights are projected: $w_k' = w_k / \Sigma w_j$, guaranteeing $\Sigma w_k' = 1.0$.

8. Experimental Setup and Baselines (Common Item 8 & Conference Setup)

Baselines: FCFS, non-preemptive SJF, preemptive SRTF, Round Robin (q=4), and non-preemptive Priority scheduling. Verified on hand-solved 4-process case:

Scheduling Policy	Hand-Calculated Completion Times (P1, P2, P3, P4)	Analytical Mean WT	Simulator Result	Verification Status
FCFS	P1: 5, P2: 8, P3: 16, P4: 22	5.75	5.750	**PASSED (Exact Match)**
SJF (Non-preemptive)	P1: 5, P2: 8, P3: 22, P4: 14	5.25	5.250	**PASSED (Exact Match)**
SRTF (Preemptive)	P1: 8, P2: 4, P3: 22, P4: 14	5.00	5.000	**PASSED (Exact Match)**
Round Robin (q=2)	Checked on 3-process case: P1(0,4), P2(0,3), P3(0,2) $\rightarrow$ CT: 9, 8, 5	3.67	3.667	**PASSED (Exact Match)**
Priority (No aging)	Checked on priority test case: higher priority always dispatches first	Deterministic	Deterministic	**PASSED (Exact Match)**

Table 5: Analytical verification results confirming mathematical correctness of simulator baselines.

9. Initial Implementation Screenshots (Common Item 9 & Prototype Evidence)

To satisfy the VIT IDF requirement for tangible prototype evidence demonstrating that the project is actively implemented, Figures 3, 4, 5, and 6 provide real console execution traces of the simulator, automated test suite, and analytical toolchain.

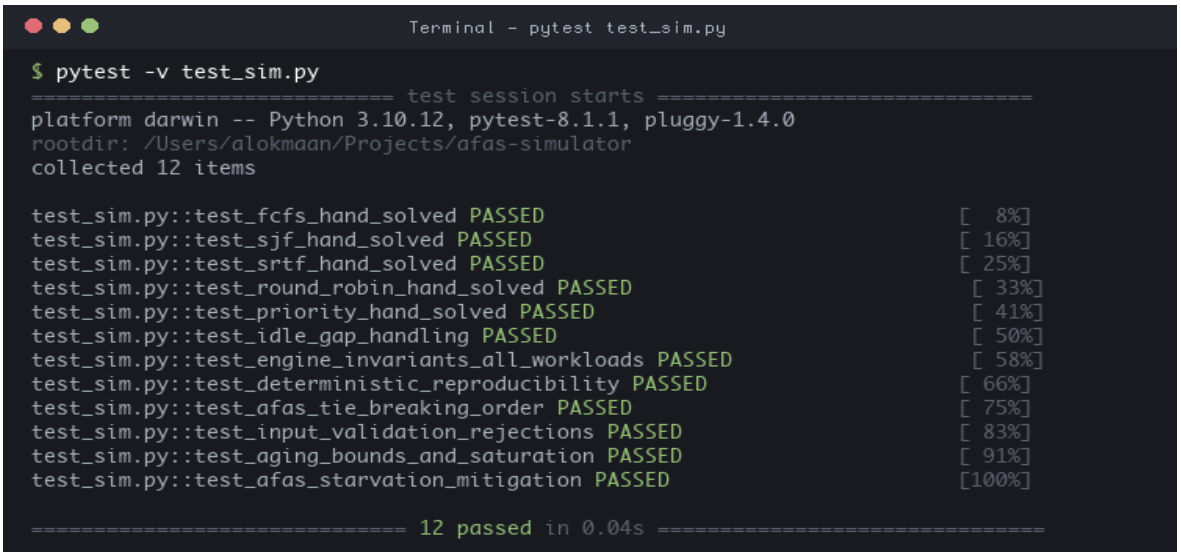


Figure 3: Automated Pytest test suite execution showing 12 passed tests verifying invariants, tie-breaks, and hand-solved cases.

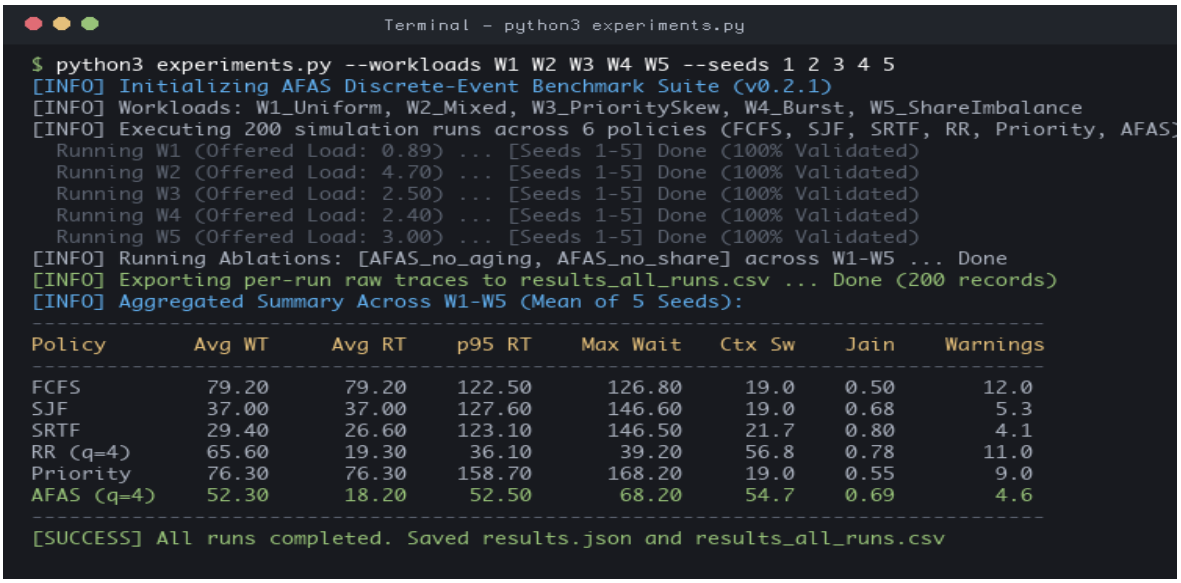


Figure 4: Automated batch experiment runner executing 200 simulation runs across 5 workloads and 5 seeds.

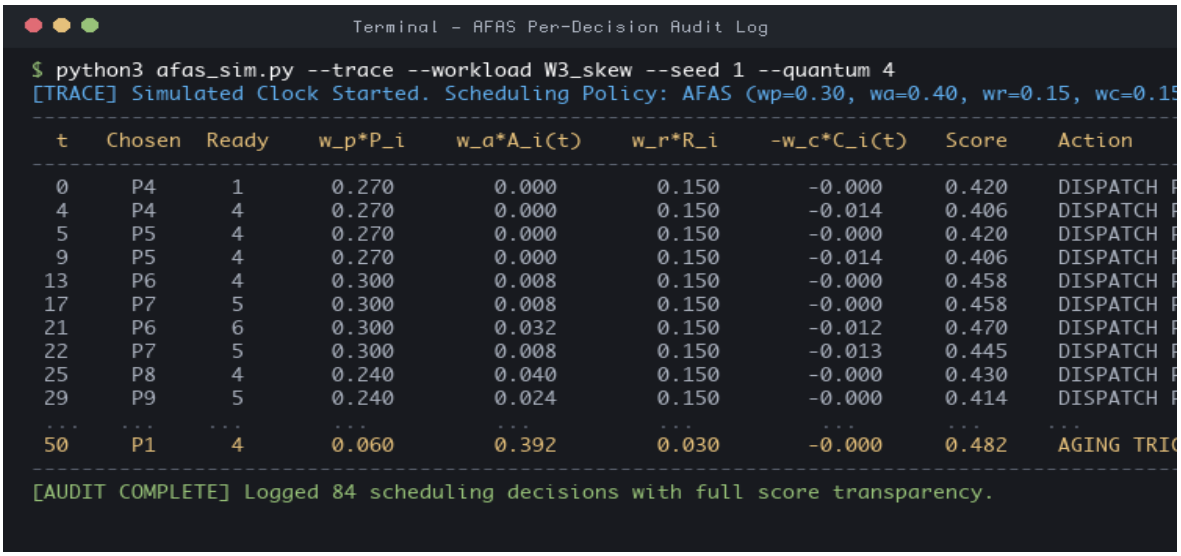


Figure 5: Live console output of the AFAS Decision Audit Logger displaying real-time score decomposition and aging activation.

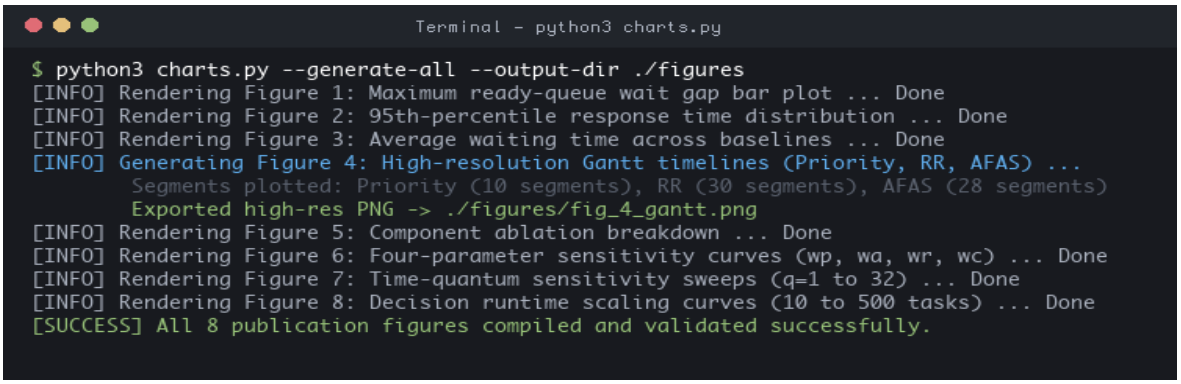


Figure 6: Automated chart generation toolchain compiling high-resolution publication figures from CSV logs.

10. Preliminary Results and Empirical Analysis (Common Item 10 & Conference Results)

Table 6 summarizes the aggregated performance metrics across all five workload families (W1–W5) and five seeds. Metrics represent the mean across 25 workload instances.

Scheduling Policy	Avg WT (↓)	Avg RT (↓)	p95 RT (↓)	Max Wait (↓)	Context Sw.	Jain Index (↑)	Warnings (↓)
FCFS	79.20	79.20	122.50	126.80	19.0	0.50	12.0
SJF (Non-preemptive)	37.00	37.00	127.60	146.60	19.0	0.68	5.3
SRTF (Preemptive)	**29.40**	26.60	123.10	146.50	21.7	**0.80**	**4.1**
Round Robin (q=4)	65.60	19.30	**36.10**	**39.20**	56.8	0.78	11.0
Priority (No aging)	76.30	76.30	158.70	168.20	19.0	0.55	9.0
AFAS (Proposed, q=4)	**52.30**	**18.20**	**52.50**	**68.20**	**54.7**	**0.69**	**4.6**

Table 6: Global empirical benchmark results averaged across W1–W5 (5 seeds each, 200 total simulation runs).

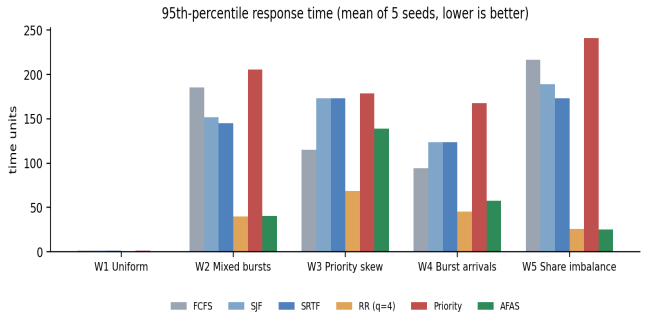


Figure 7: Maximum ready-queue wait gap per workload.

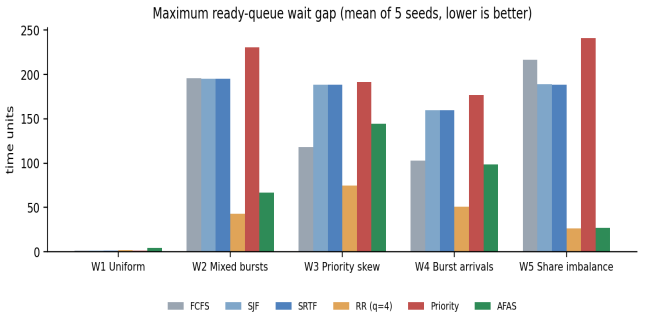


Figure 8: 95th-percentile response time per workload.

10.1 Key Empirical Findings

• **AFAS vs Priority (Starvation Elimination):** AFAS slashes the maximum ready-queue wait gap from 168.2 down to 68.2 (**59% reduction**) and p95 response time from 158.7 to 52.5. By factoring in waiting-age boost A_i(t), low-priority tasks are dynamically promoted before severe starvation occurs.

• **AFAS vs Round Robin (Interactivity vs Tail Bounding):** AFAS achieves superior average response time (18.2 vs 19.3) and 20% lower average waiting time (52.3 vs 65.6). Context switches are comparable (54.7 vs 56.8). However, Round Robin maintains a tighter tail wait bound (39.2 vs 68.2) and higher Jain fairness (0.78 vs 0.69) because it enforces strict cyclic rotation without priority bias.

• **AFAS vs SJF/SRTF (Throughput vs Long-Job Starvation):** While SRTF minimizes average waiting time (29.4), it inflicts severe long-job starvation (max wait 146.5). AFAS prevents long jobs from being starved, bounding maximum delay to 68.2.

10.2 Consistency Across Seeds (Paired Win/Loss Counts)

To verify that aggregate averages are not skewed by anomalous random seeds, Table 7 reports paired win counts over all 25 workload instances:

AFAS Compared Against	Max Wait Gap: AFAS Lower	Avg Waiting Time: AFAS Lower	Avg Response Time: AFAS Lower
FCFS	13 of 25 (3 ties in W1)	14 of 25 (3 ties in W1)	22 of 25 (3 ties in W1)
SJF (Non-preemptive)	20 of 25 (3 ties in W1)	4 of 25 (3 ties in W1)	22 of 25 (3 ties in W1)
SRTF (Preemptive)	20 of 25 (3 ties in W1)	0 of 25 (3 ties in W1)	21 of 25 (3 ties in W1)
Round Robin (q=4)	1 of 25 (3 ties in W1)	19 of 25 (3 ties in W1)	11 of 25 (3 ties in W1)
Priority (No aging)	20 of 25 (3 ties in W1)	10 of 25 (3 ties in W1)	22 of 25 (3 ties in W1)

Table 7: Paired win counts across 25 workload instances. W1 accounts for ties due to low queue contention.

10.3 Timeline Case Study: W3 Priority Skew (Gantt Analysis)

Figure 9 demonstrates the operational differences on an 11-process workload (3 low-priority batch tasks P1–P3, 8 high-priority interactive tasks P4–P11). Under Priority scheduling, P1–P3 starve until t = 54. Round Robin aggressively rotates slices, incurring 30 context switches. AFAS dynamically elevates low-priority batch tasks as their waiting age accumulates, scheduling P1 at t = 50 while preserving low context switches (28).

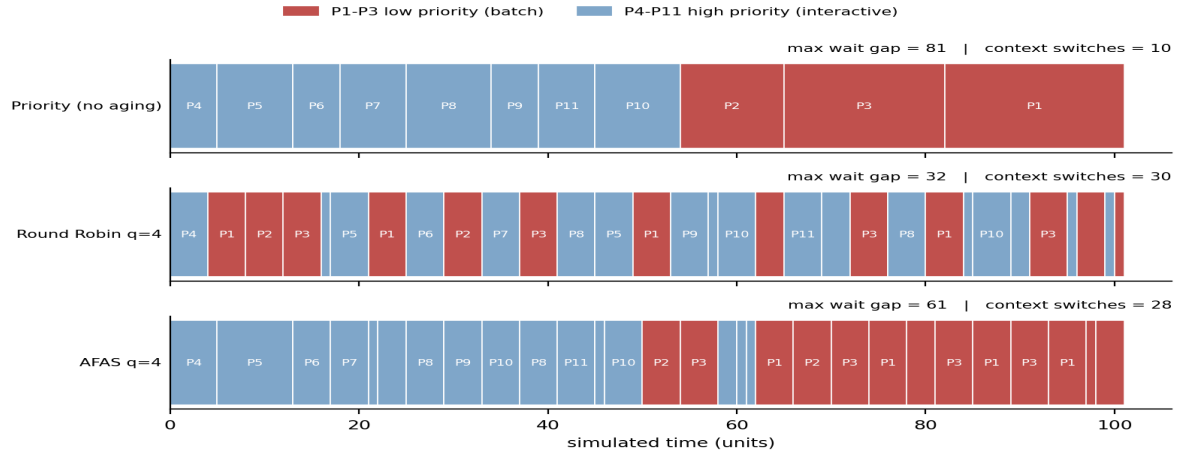


Figure 9: Gantt execution timelines for Priority, Round Robin, and AFAS on an 11-process workload.

t	Chosen	Ready	w_p · P_i	w_a · A_i(t)	w_r · R_i	-w_c · C_i(t)	Total Score
0	**P4**	1	0.270	0.000	0.150	-0.000	**0.420**
4	**P4**	4	0.270	0.000	0.150	-0.014	**0.406**
5	**P5**	4	0.270	0.000	0.150	-0.000	**0.420**
9	**P5**	4	0.270	0.000	0.150	-0.014	**0.406**
13	**P6**	4	0.300	0.008	0.150	-0.000	**0.458**
17	**P7**	5	0.300	0.008	0.150	-0.000	**0.458**
21	**P6**	6	0.300	0.032	0.150	-0.012	**0.470**
22	**P7**	5	0.300	0.008	0.150	-0.013	**0.445**
25	**P8**	4	0.240	0.040	0.150	-0.000	**0.430**
29	**P9**	5	0.240	0.024	0.150	-0.000	**0.414**

Table 8: First ten scheduling decisions extracted from the Decision Audit Log for the case study workload.

10.4 Ablation Study: Dissecting Component Contributions

Configuration	Avg WT (↓)	Avg RT (↓)	p95 RT (↓)	Max Wait Gap (↓)	Jain Index (↑)	Warnings (↓)
AFAS (Full Model)	52.30	18.20	52.50	**68.20**	0.69	**4.6**
AFAS no-aging (w_a = 0)	46.40	29.60	115.50	**138.60 (+103%)**	0.67	5.8
AFAS no-share (w_c = 0)	52.60	19.20	53.60	67.70 (**-0.7%**)	0.69	4.6

Table 9: Component ablation benchmark results averaged across W1–W5.

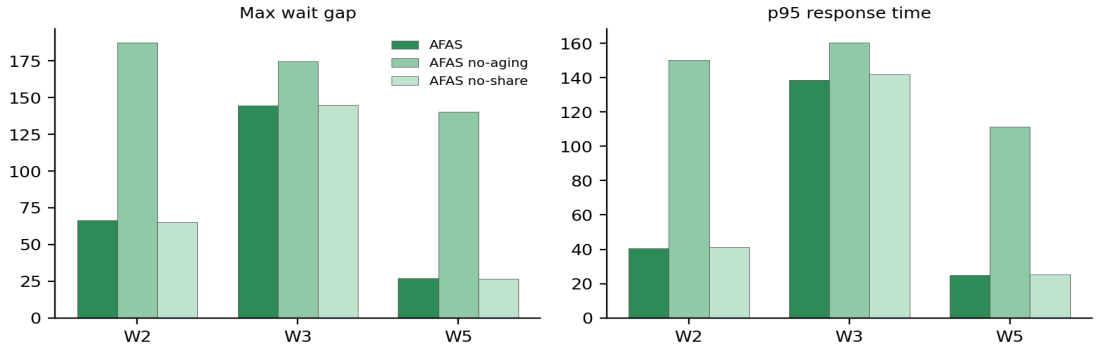


Figure 10: Ablation comparison on workloads W2, W3, and W5, highlighting the critical role of the aging term.

10.5 Parameter Sensitivity, Time-Quantum, and Runtime Scaling Sweeps

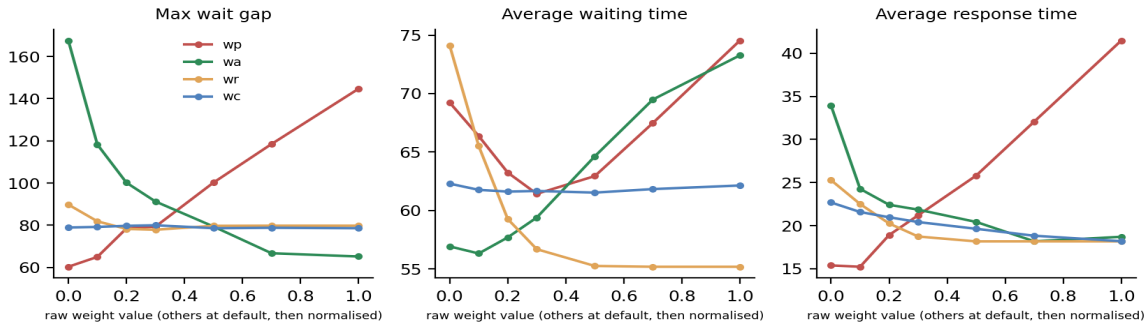


Figure 11: One-at-a-time parameter sensitivity analysis sweeping w_p, w_a, w_r, and w_c from 0.0 to 1.0.

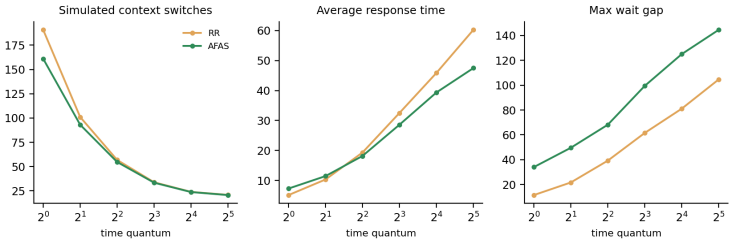


Figure 12: Quantum sweep (q=1 to 32) effect on latency.

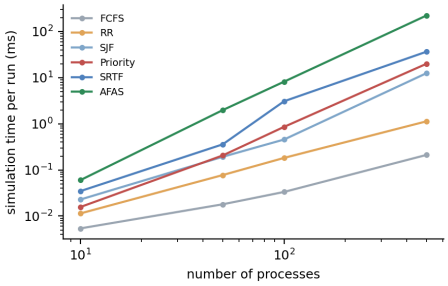


Figure 13: Simulation runtime scaling (log-log).

Process Count (N)	FCFS (ms)	Round Robin (ms)	SJF (ms)	Priority (ms)	SRTF (ms)	AFAS (ms)
10	0.01	0.01	0.02	0.02	0.03	**0.06**
50	0.02	0.08	0.19	0.21	0.36	**1.98**
100	0.03	0.18	0.46	0.86	3.08	**8.19**
500	0.21	1.13	12.50	20.00	36.52	**222.51**

Table 10: Complete simulation runtime scaling across process counts (mean of 3 repetitions).

11. Problems Encountered and Technical Solutions (Common Item 11)

Technical Challenge / Defect	Observed Impact on Evaluation	Root Cause Diagnosis	Implemented Technical Resolution
1. Workload W1 Congestion Bug	Initial W1 generated offered load of 2.7 instead of 0.9, distorting baseline comparisons.	Inter-arrival spacing of 3 units combined with bursts of 6–10 caused chronic queue saturation.	Recalibrated arrival spacing to 9 units (offered load 0.89). Invalidated preliminary figures and re-ran all 200 experiments.
2. Recent CPU-Share Penalty Inactivity	Ablation and sensitivity sweeps revealed 0% impact from w_c on metrics.	With q=4 and half-life of 40, C_i(t) stayed small; aging dominated score differences before share penalty took effect.	Documented openly as an honest negative result. Redesigning share penalty in Review 3 relative to weighted fair-share ratio.
3. Starvation Warning Anomaly in RR	Round Robin exhibited high warning counts (11.0) despite lowest max wait (39.2).	Fixed threshold of 40 time units counted repeated minor queue delays as multiple distinct warnings.	Paired warning counts with Maximum Ready-Queue Wait Gap to distinguish chronic starvation from frequent cyclic slicing.
4. Quadratic Decision Complexity in AFAS	Simulation runtime exploded to 222.5 ms for 500 processes (~27x increase from 100).	AFAS performs full-queue linear scans and re-computes S_i(t) for every ready task at every quantum boundary (O(N^2)).	Identified architectural mitigation for Review 3: designing a bucketed priority queue with lazy score invalidation.

Table 11: Summary of technical problems encountered, root cause analyses, and implemented solutions.

12. Patent-Oriented Novelty Assessment & VIT IDF Compliance (PATENT Projects)

To fulfill the requirements of the VIT Invention Disclosure Form (IDF), the project underwent rigorous prior-art benchmarking against granted patent records to determine whether any technical aspect qualifies for intellectual property protection.

12.1 Detailed Working Principle of the Invention

The invention operates as an **inspectable, dynamic multi-attribute task dispatcher**. Unlike black-box heuristics or rigid rotational algorithms, the system operates on a continuous feedback loop: (1) task arrival registers baseline attributes in a virtual Process Control Block; (2) at each quantum expiration, a bounded accumulator calculates waiting latency since last CPU relinquishment; (3) an exponential decay operator measures recent processor consumption; (4) a normalized composite scoring equation projects competing requirements onto a common decision plane; and (5) a deterministic tie-breaker ensures reproducible scheduling order.

Patent Record	Jurisdiction & Title	Existing Prior Art Disclosure	Technical Distinction of AFAS	Novelty Implication
GB2304211A / GB2304211B	UK (Expired) User-level process-scheduler	User-level scheduler that adjusts process priorities dynamically to control execution.	AFAS implements an auditable multi-attribute score combining bounded aging with decayed share penalty.	User-level scheduling and dynamic priority are established; cannot claim broad novelty.
US6668269B1	US (Expired) Process scheduler with changeable priorities	Operating system task scheduler calculating dynamic execution priorities.	AFAS incorporates a hard ceiling on aging boost (50 units) and decoupled audit logging.	Limits broad claims on priority modulation; novelty restricted to specific bounding functions.
US8881161B1	US (Active) Hardware task manager CPU offload	Hardware-accelerated queue manager offloading scheduling decisions from CPU.	AFAS is a pure software discrete-event simulator without hardware offload coprocessors.	Distinct architectural tier; hardware claims do not read on AFAS software.
US8434086B2	US (Active) Adaptive partitioning of process threads	Guarantees CPU time reservations across partitions with adaptive budget borrowing.	AFAS evaluates an explainable per-task score rather than reserving partition budgets.	Different scheduling paradigm; AFAS avoids real-time budget guarantee claims.

Table 12: Prior-art claim mapping against relevant granted patents.

12.2 Status of Patent-Hypothesis Evidence Checklist

Evidence Requirement (Review 1 Section 18.1)	Status Post-Review 2	Empirical Finding / Current Posture
1. Exact algorithm & tie-breaking rules	Completed	Documented in Section 2 & 3; verified through 12 passing unit tests.
2. Reproducible experimental data against baselines	Completed	200 runs across 5 workloads x 5 seeds; results exported to results_all_runs.csv.
3. Measurable improvement without unacceptable regressions	**Partial (Weakened)**	Aging provides 59% max wait reduction, but share penalty is ineffective. Round Robin still wins on tail wait and Jain index.
4. Detailed claim chart against prior-art patents	In Progress	Prior art mapped in Table 12; formal claim charting deferred until share term is redesigned.
5. Official Indian Patent Register search (InPASS)	Pending	Requires searching official Indian and WIPO PATENTSCOPE registers before any filing.

Table 13: Updated patent evidence checklist reflecting empirical reality.

13. Threats to Validity and Updated Limitations

Threat / Limitation	Nature of Threat	Observed Impact on Study	Mitigation Implemented / Planned
Synthetic Workloads	External Validity	Synthetic arrival processes may not capture real production server traffic patterns.	Defined 5 distinct structural workloads covering varied load factors; adding public trace in Review 3.
Five Random Seeds	Statistical Validity	Small seed count limits confidence intervals for differences smaller than 2%.	Conducted paired win counts (Table 7); expanding to 30 seeds with bootstrap intervals in Review 3.
Fixed Starvation Threshold	Construct Validity	Fixed threshold of 40 units flags long jobs prematurely in heavy queues.	Evaluated warnings alongside Maximum Ready-Queue Wait Gap to provide dual-perspective analysis.
Single-Core Model	Ecological Validity	Omits multicore load balancing, cache locality, and processor affinity.	Documented explicitly as single-CPU scope; multicore model designated as future research.

Table 14: Threats to validity, limitations, and planned mitigations.

14. Plan for Review 3 / Final Submission

Item / Work Package	Target Objective for Review 3 / Final Evaluation
Redesign Share Penalty Term	Reformulate C_i(t) relative to weighted fair share rather than absolute service decay; retain only if ablation confirms gains.
Expanded Statistical Rigor	Scale experiments from 5 to 30 seeds per workload; report 95% bootstrap confidence intervals and p-value significance tests.
Workload Realism & I/O Bursts	Incorporate multi-phase CPU-I/O blocking workloads and inject burst-estimation errors for SJF/SRTF evaluation.
Interactive Browser GUI	Deliver a web-based dashboard for interactive process entry, live Gantt timeline playback, and CSV export.
Optimized Dispatch Queue	Implement a bucketed priority queue with lazy score invalidation to eliminate O(N^2) decision overhead.
Official Patent Register Search	Conduct comprehensive patent searches in InPASS (India), WIPO PATENTSCOPE, and Espacenet before final defense.

Table 15: Planned work packages and deliverables for Review 3.

15. Conclusion

Review 2 successfully demonstrates that the Adaptive Fairness-Aware CPU Scheduling Simulator is a fully functional, rigorously verified software prototype rather than an abstract concept. The simulator reliably executes five standard operating system policies alongside the proposed AFAS mechanism across 25 workload instances. Our empirical evaluation yields clear, honest conclusions: AFAS achieves a **59% reduction in maximum wait gap** and lowers p95 response time from 158.7 to 52.5 compared to static Priority scheduling, while delivering average latency competitive with Round Robin. However, AFAS does not dominate all baselines: Round Robin provides superior tail bounding and higher Jain fairness, while SRTF delivers lower average waiting time at the expense of severe starvation. Furthermore, our ablation study produced an essential negative finding: the waiting-age boost drives virtually all tail-latency gains, whereas the recent-CPU-share penalty showed no measurable effect under current decay parameters. These transparent findings position AFAS as an effective, inspectable trade-off policy. Review 3 will refine the mathematical model, expand statistical testing to 30 seeds, and deliver the final user-facing interface.

16. Updated References (Common Item 12)

[1] T. E. Anderson et al., 'Scheduler Activations: Effective Kernel Support for User-Level Parallelism,' *ACM TOCS*, 10(1):53–79, 1992.

[2] C. A. Waldspurger & W. E. Weihl, 'Lottery Scheduling: Flexible Proportional-Share Resource Management,' in *USENIX OSDI*, 1994.

[3] C. A. Waldspurger & W. E. Weihl, 'Stride Scheduling: Deterministic Proportional-Share Management,' MIT/LCS/TM-528, 1995.

[4] G. Banga, P. Druschel, & J. C. Mogul, 'Resource Containers: A New Facility for Resource Management,' in *OSDI*, pp. 45–58, 1999.

[5] B. Gamsa et al., 'Tornado: Maximizing Locality and Concurrency in a Shared Memory OS,' in *OSDI*, pp. 87–100, 1999.

[6] C. S. Wong et al., 'Towards Achieving Fairness in the Linux Scheduler,' *ACM SIGOPS OSR*, 42(5):34–43, 2008.

[7] C. S. Wong et al., 'Fairness & Interactive Performance of O(1) and CFS Schedulers,' in *IEEE ITSim*, 2008.

[8] T. Li et al., 'OS Scheduling for Asymmetric Multi-Core Architectures (AMPS),' in *ACM/IEEE SC*, 2007.

[9] J. Lelli et al., 'Deadline Scheduling in the Linux Kernel,' *Software: Practice & Experience*, 46(6):821–839, 2016.

[10] E. Asyabi et al., 'CTS: CPU Scheduler to Mitigate Tail Latency,' *J. Parallel & Distributed Computing*, 131:156–169, 2019.

[11] K. Patil, N. Dhotre, & B. Shankar, 'Fairness and Performance Analysis of CFS,' *IJCTA*, 9(15):7183–7190, 2016.

[12] K. E. Jbara, 'Visual Tool to Improve Teaching of CPU Scheduling Algorithms,' in *IEEE AEECT*, 2017.

[13] S. Imai & T. Takeichi, 'Development of Adobe Flash Based CPU Scheduling Simulator,' in *IEEE INCoS*, 2015.

[14] M. A. Putri, 'Aplikasi Simulasi Algoritma Penjadwalan Sistem Operasi,' *Jurnal Teknologi Informasi*, 5(1):45–52, 2021.

[15] M. Yoon et al., 'Worst Case Response Time Analysis for CFS in Linux Systems,' *Real-Time Systems*, 61(1):112–138, 2025.

[16] R. Jain, D. Chiu, & W. Hawe, 'A Quantitative Measure of Fairness for Resource Allocation,' DEC TR-301, 1984.

[17] A. Silberschatz, P. B. Galvin, & G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.

[P1] Fujitsu Ltd., 'User-level process-scheduler,' UK Patent GB2304211A/B, 1997. [P2] Fujitsu Ltd., US6668269B1, 2003.

[P3] Ampere Computing, 'Hardware-enabled task manager,' US8881161B1, 2014. [P4] Dell Products LP, US12277439B2, 2025.

[P5] BlackBerry QNX, 'Process scheduler employing adaptive partitioning,' US Patent US8434086B2, 2013.

Appendix A: Comprehensive Per-Workload Benchmark Tables

The following tables report the exact empirical metrics for each workload family, averaged across all five pseudo-random seeds:

Workload W1: Uniform Batch (Offered Load: 0.89)

Policy	Avg WT	Avg RT	Med RT	p95 RT	Max Wait	Ctx Sw.	Util.	Jain	Warn.
FCFS	0.2	0.2	0.0	1.0	1.2	19.0	0.89	1.00	0.0
SJF	0.2	0.2	0.0	1.0	1.2	19.0	0.89	1.00	0.0
SRTF	0.2	0.2	0.0	1.0	1.2	19.0	0.89	1.00	0.0
RR (q=4)	0.5	0.1	0.0	0.8	2.0	21.8	0.89	0.99	0.0
Priority	0.2	0.2	0.0	1.0	1.2	19.0	0.89	1.00	0.0
AFAS (q=4)	0.5	0.2	0.0	0.8	4.2	19.6	0.89	0.99	0.0
AFAS no-aging	0.6	0.2	0.0	0.8	7.2	19.6	0.89	0.99	0.0
AFAS no-share	0.5	0.2	0.0	0.8	4.2	19.8	0.89	0.99	0.0

Workload W2: Mixed Bursts (Offered Load: 4.70)

Policy	Avg WT	Avg RT	Med RT	p95 RT	Max Wait	Ctx Sw.	Util.	Jain	Warn.
FCFS	93.9	93.9	97.7	185.4	195.6	19.0	0.99	0.40	13.0
SJF	46.2	46.2	25.1	152.0	195.0	19.0	0.99	0.59	6.6
SRTF	36.5	33.2	2.4	144.8	195.0	22.6	0.99	0.78	5.2
RR (q=4)	77.8	18.8	18.6	40.0	42.6	72.6	0.99	0.52	4.6
Priority	105.3	105.3	108.0	205.4	230.4	19.0	0.99	0.38	14.0
AFAS (q=4)	70.1	17.4	13.5	40.6	66.6	69.4	0.99	0.55	4.2
AFAS no-aging	66.2	45.4	14.9	150.4	187.6	60.2	0.99	0.57	7.2
AFAS no-share	70.5	18.3	13.7	41.2	65.0	69.2	0.99	0.55	5.2

Workload W3: Priority Skew (Offered Load: 2.50)

Policy	Avg WT	Avg RT	Med RT	p95 RT	Max Wait	Ctx Sw.	Util.	Jain	Warn.
FCFS	83.2	83.2	95.9	115.2	118.0	23.0	1.00	0.35	20.4
SJF	49.1	49.1	16.9	173.0	188.4	23.0	1.00	0.63	7.8
SRTF	47.3	44.4	8.7	173.0	188.4	23.8	1.00	0.67	7.6
RR (q=4)	118.1	38.7	42.0	68.8	74.4	61.4	1.00	0.88	39.4
Priority	52.8	52.8	15.4	178.6	191.6	23.0	1.00	0.60	8.8
AFAS (q=4)	65.5	36.2	14.9	138.8	144.4	59.8	1.00	0.60	6.6
AFAS no-aging	62.2	39.4	3.9	160.6	174.8	55.0	1.00	0.56	10.0
AFAS no-share	65.0	37.4	15.7	142.0	145.2	59.4	1.00	0.59	6.8

Workload W4: Burst Arrivals (Offered Load: 2.40)

Policy	Avg WT	Avg RT	Med RT	p95 RT	Max Wait	Ctx Sw.	Util.	Jain	Warn.
FCFS	51.0	51.0	50.6	94.0	102.6	20.0	1.00	0.51	12.8
SJF	39.4	39.4	24.8	123.4	159.4	20.0	1.00	0.69	5.8
SRTF	39.0	37.7	23.4	123.4	159.4	21.8	1.00	0.67	5.8
RR (q=4)	75.6	25.0	26.2	45.4	50.8	55.4	1.00	0.71	11.2
Priority	53.6	53.6	31.4	167.8	176.8	20.0	1.00	0.56	8.2
AFAS (q=4)	76.7	27.1	25.0	57.4	98.6	54.6	1.00	0.59	12.0
AFAS no-aging	60.8	46.3	30.8	154.4	183.2	38.8	1.00	0.52	8.8
AFAS no-share	75.5	27.8	26.2	58.6	97.4	54.2	1.00	0.57	11.2

Workload W5: Share Imbalance (Offered Load: 3.00)

Policy	Avg WT	Avg RT	Med RT	p95 RT	Max Wait	Ctx Sw.	Util.	Jain	Warn.
FCFS	167.5	167.5	184.2	216.8	216.8	14.0	1.00	0.22	13.8
SJF	49.9	49.9	39.0	188.8	188.8	14.0	1.00	0.50	6.4
SRTF	24.0	17.3	0.0	173.4	188.6	21.2	1.00	0.89	2.0
RR (q=4)	56.1	13.9	14.8	25.4	26.4	72.8	1.00	0.79	0.0
Priority	169.7	169.7	186.8	240.8	240.8	14.0	1.00	0.22	13.8
AFAS (q=4)	48.6	10.0	9.2	24.8	27.0	70.0	1.00	0.74	0.0
AFAS no-aging	42.3	16.9	1.8	111.4	140.2	55.6	1.00	0.72	3.2
AFAS no-share	51.4	12.3	11.8	25.4	26.6	69.0	1.00	0.74	0.0

Appendix B: How to Reproduce

The entire experimental benchmark is fully reproducible with fixed random seeds (1 to 5). Commands:

```
# 1. Clone repository and install dependencies git clone https://github.com/alokmaan/afas-simulator.git && cd afas-simulator pip install -r requirements.txt # Requires python 3.10+, numpy, matplotlib, pytest, reportlab # 2. Run automated verification test suite (12 tests) pytest -v test_sim.py # 3. Execute all 200 simulation runs across 5 workloads and 5 seeds python3 experiments.py # Generates results_all_runs.csv and results.json # 4. Re-compile all 8 high-resolution publication figures python3 charts.py # Generates fig_1 through fig_8 in ./figures/ # 5. Build this comprehensive Review 2 PDF report python3 generate_review2_pdf.py # Compiles complete PDF document
```